What is new in log4j version 1.3?

by Ceki Gülcü and Mark Womack

ApacheCon US 2003, 18 Nov 2003



Outline of our presentation

1. Short introduction to log4j.
2. A new logging dimension in the form of logging domains
3. Support for custom conversion words in `PatternLayout`
4. Strategy based log file rollover
5. Highly extensible XML configuration files (`JoranConfigurator`)
6. Significant improvements in speed and memory management
7. Plugins, Receivers, and Watchdogs...
8. Interoperability with log4j-like logging frameworks in other languages
9. Spectacular improvements to Chainsaw
10. Conclusion

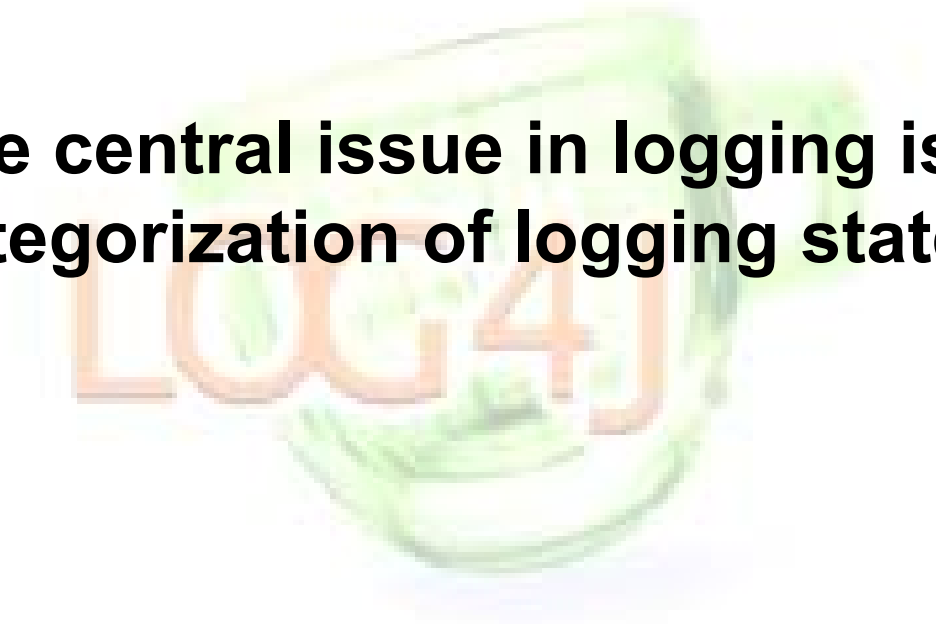
Logging, a definition

- Logging is a low-tech method for debugging applications.
- Logging can also be viewed as an *auditing* tool.

Basic features of any logging library are:

1. control over which logging statements are enabled or disabled,
2. manage output destinations,
3. manage output format.

The central issue in logging is the categorization of logging statements.



Loggers, a definition.

- Loggers are *named entities* that follow the *hierarchical naming* rule.
- Example:
  - The logger named com is the direct parent of the logger named `com.foo` and an ancestor of the logger named `com.foo.bar`.

Log4j Levels

- Loggers *may* be assigned levels. The set of possible levels, that is DEBUG, INFO, WARN, ERROR and FATAL are defined in the org.apache.log4j.Level class.

- If a given logger is not assigned a level, then it **inherits** one from its closest ancestor with an assigned level.

- The root logger resides at the top of the logger hierarchy. It always exists and always has an assigned level.

Selection Rule:

- A logging request is said to be *enabled* if its effective level is higher than or equal to the level of its logger. Otherwise, the request is said to be *disabled*.

Reminder: A logger without an assigned level will inherit one from the hierarchy.

Putting it together:

```
// get a logger instance named "com.foo"
Logger logger = Logger.getInstance("com.foo");

// Now set its level.
logger.setLevel(Level.INFO);

Logger barLogger = Logger.getInstance("com.foo.Bar");

// This request is enabled, because WARN >= INFO.
logger.warn("Low fuel level.");

// This request is disabled, because DEBUG < INFO.
logger.debug("Starting search for nearest gas station.");

// The logger instance barLogger, named "com.foo.Bar",
// will inherit its level from the logger named
// "com.foo" Thus, the following request is enabled
// because INFO >= INFO.
barLogger.info("Located nearest gas station.");

// This request is disabled, because DEBUG < INFO.
barLogger.debug("Exiting gas station search");
```

Log4j *appenders*

- Output targets are implemented by appenders. Appenders exists for java.io.Writer, the console, files, Unix syslog, JMS, databases, SMTP, and more.
- Multiple appenders can be attached to any logger.
- Appenders follow the logger hierarchy in an *additive* fashion.

Layouts & ObjectRenderers

- Users can customize the output of an appender by associating with it a Layout.
- The PatternLayout formats a logging event according to a conversion pattern.
- Logging output can be formatted also in HTML or XML among other formats.
- It is also possible to format the output by message object *type* by registering an appropriate ObjectRenderer for a given type.

Configuration

- Log4j allows both programmatical and script-based configuration.
- Configuration scripts can be expressed as key=value pairs or in XML format.

MyApp example 1 of 2

```
import com.foo.Bar;

// Import log4j classes.
import org.apache.log4j.Logger;
import org.apache.log4j.BasicConfigurator;

public class MyApp {

  // Define a static logger variable so that it references
  // the Logger instance named "MyApp".
  static Logger logger = Logger.getInstance(MyApp.class);

  public static void main(String[] args) {

    // Set up a simple configuration that logs on the console.
    BasicConfigurator.configure();

    logger.info("Entering application.");
    Bar bar = new Bar();
    bar.doIt();
    logger.info("Exiting application.");
  }
}
```

MyApp first example (2 of 2)

```
package com.foo;
import org.apache.log4j.Logger;

public class Bar {
  static Logger logger = Logger.getInstance(Bar.class);

  public void doIt() {
     logger.debug("Did it again!");
  }
}
```

The output of MyApp is:

```
0 [main] INFO MyApp - Entering application.
36 [main] DEBUG com.foo.Bar - Did it again!
51 [main] INFO MyApp - Exiting application.
```

Second MyApp example (1 of 4)

```
// Import log4j classes.
import org.apache.log4j.Logger;
import org.apache.log4j.PropertyConfigurator;

public class MyApp {

  // Define a static logger variable so that it references
  // the Logger instance named "MyApp".
  static Logger logger = Logger.getInstance(MyApp.class);

  public static void main(String[] args) {

    // Set up a simple configuration that logs on the console.
    PropertyConfigurator.configure(args[0]);

    logger.info("Entering application.");
    Bar bar = new Bar();
    bar.doIt();
    logger.info("Exiting application.");
  }
}
```

Second MyApp example (2 of 4)

Here is a equivalent configuration file equivalent to invoking BasicConfigurator.configure method:

```
# Set root logger level to DEBUG and its only
# appender to A1.
log4j.rootLogger=DEBUG, A1

# A1 is set to be a ConsoleAppender which outputs to
# System.out.
log4j.appender.A1=org.apache.log4j.ConsoleAppender

# A1 uses PatternLayout.
log4j.appender.A1.layout=org.apache.log4j.PatternLayout
log4j.appender.A1.layout.ConversionPattern=%-4r [%t] %-5p %c
%x - %m%n
```

Second MyApp example (3 of 4)

Here is a configuration file that suppresses the debugging output from any component in the com.foo package.

```
log4j.rootLogger=DEBUG, A1
log4j.appender.A1=org.apache.log4j.ConsoleAppender
log4j.appender.A1.layout=org.apache.log4j.PatternLayout

# Print the date in ISO 8601 format
log4j.appender.A1.layout.ConversionPattern=%d %p %m%n

# Print only messages of level WARN or above in
# the package com.foo.
log4j.logger.com.foo=WARN
```

MyApp configured with this file results in:

```
2001-03-12 14:07:41,508 INFO Entering application.
2001-03-12 14:07:41,529 INFO Exiting application.
```

Part II – New features in 1.3

<WARNING>

Log4j 1.3 is still in alpha stage. The following content might not be accurate by the time 1.3 rolls out officially.

</WARNING>

Logging domains (new in 1.3)

- Domains possess the following properties.
  - They are arranged in a named hierarchy.
  - They have levels.
  - Appenders can be attached to domains.
- A *logger* encapsulates a *domain*.

Sub-domains

- A *sub-domain* is almost the same as a domain.
- A sub-domain differs from domain in that it is are marked as such.
- A logger can encapsulate a domain *or* a domain plus a sub-domain.

Sub-domains (continued)

- When a logger encapsulates a sub-domain and a domain, the sub-domain is consulted first.
- The sub-domain is asked how the logging request of a certain level should be handled.
- The sub-domain's reply contains two components, a "local appender" part and a "further processing" part.

Sub-domain's reply

The sub-domain's six possible responses:

#	Local appenders	Further processing
1	Append locally	and let the next (main) domain decide about further processing.
2	Do not append locally	and let the next (main) domain decide about further processing.
3	Append locally	and consider request *enabled* for the main domain.
4	Do not append locally	and consider request *enabled* for the main domain.
5	Append locally	and consider request *disabled* for the main domain.
6	Do not append locally	and consider request *disabled* for the main domain.

Use cases for each answer:

Answer 1) Let requests in a sub-domain to be output to appenders in a sub-domain without interfering with processing by main domain.

Answer 2) The logical complement of answer 1).

Answer 3) Force the request to be enabled due to inclusion in sub-domain. Also append to appenders in sub-domains if any.

Answer 4) Force the request to be enabled due to inclusion in sub-domain. Do not append to appenders in sub-domains if any.

Answer 5) Force the request to be disabled due to inclusion in sub-domain. Also append to appenders in sub-domains if any.

Answer 6) Force the request to be disabled due to inclusion in sub-domain. Do not append to appenders in sub-domains if any.

Abstract view of sub-domains

- Sub-domains allow the developer to log through a second dimension.

- For example, it will be possible to log by **locality** and also by **functionality**.

Enhancements to PatternLayout

Conversion specifiers are now composed of *conversion words* instead of a single *conversion character*. Here is a sample pattern that uses conversion words:

```
%relative [%thread] %level %logger - \
    %message%n
```

Note that the above pattern converter is strictly equivalent to:

```
  %r [%t] %p %c - %m%n
```

Learning new conversion words dynamically

PatternLayout is now capable of learning new conversion words dynamically.

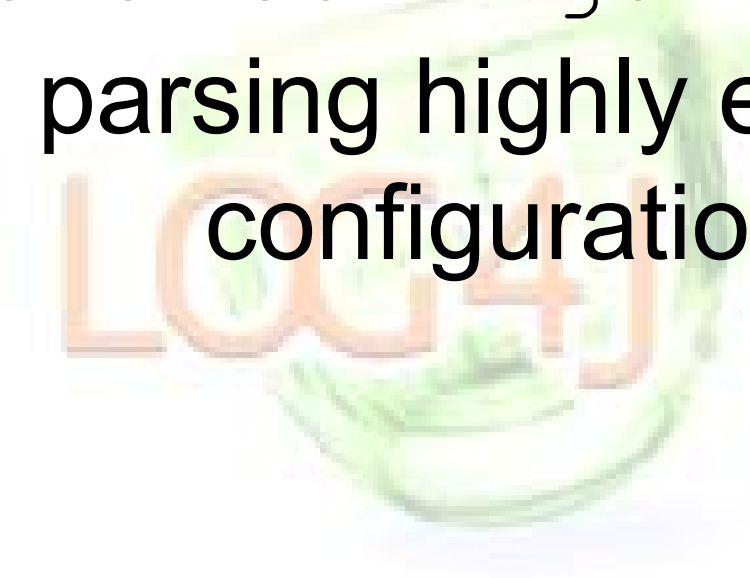
Adding a new conversion word to a `PatternLayout` instance:

```
PatternLayout pl = new PatternLayout();
pl.addConversionRule("priority",
      "org.apache.log4j.LevelPatternConverter");
pl.setConversionPattern(
                  "%-5priority [%thread] - %m%n");
pl.activateOptions();
```

Strategy based rollover of log files

- Idea: use composition instead of inheritance.
- Rolling strategy
  - deals with what to roll and how
- Triggering strategy
  - Deals with when to roll

`JoranConfigurator` for parsing highly extensible configuration files



Joran architecture

- Joran uses rules to interpret configuration files.
  - Similar conceptually to commons-digester.
- Rules consist of the association of a *pattern* and an *action*.

On patterns

- Patterns can be *exact matches* and *wildcard matches*.
- The pattern “`a/b`” will match a `<b>` element nested within an `<a>` element but not a `<c>` element even if nested within `<b>`.
- It is also possible to match a particular tag, regardless of its nesting, by using the "*" wildcard character. The pattern "`*/a`" will match an `<a>` element at anywhere within the document.

On *actions*

- Actions are called by Joran when a pattern match occurs for a particular element in the document.
- Actions work within a context, where they can save data so that subsequent actions can retrieve them.

JoranConfigurator

- Joran will consider implicit action only if no pattern could be matched.
  - E.g. `NestedComponentImplicitAction`
- `JoranConfigurator` is just an assembly of rules.
- `JoranConfigurator` has rules to learn new rules.
  - Your imagination is the limit.

Improvements in speed and memory management

- `ReaderWriterLocks`
  - Less thread contention when accessing loggers.
- Hierarchy-walk free decision making (to be implemented)
- Layouts write directly to output streams

  => much fewer intermediate Strings

Plugins

- Allows developers to add new functionality to “plug into” log4j. Just implement the defined Plugin interface.
- Attached to a LoggerRepository, so plugins can affect everything within a repository.
- Multiple plugins can be attached to a repository.

Plugins Continued

- Can be configured via configuration files
  - DOMConfigurator – XML format
  - PropertyConfigurator – not currently supported.
- Can be configured programmatically
  - PluginRegistry used to start, stop, restart plugins.

Plugin Configuration

XML Configuration:

```
<plugin class="org.apache.log4j.net.SocketReceiver">
   <param name="port" value="4040"/>
</plugin>
```

Programmatic Configuration:

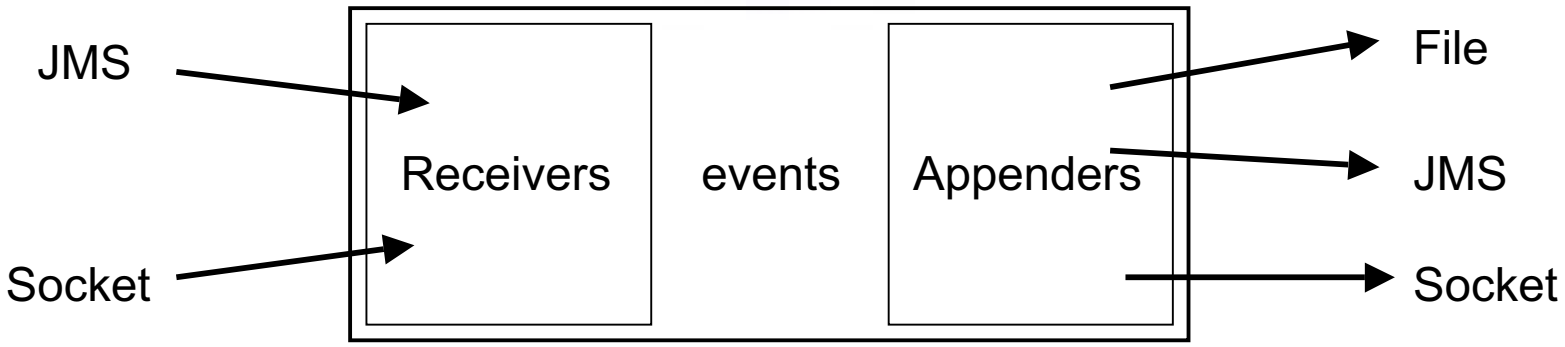
```
SocketReceiver receiver = new SocketReceiver(4040);
PluginRegistry.startPlugin(receiver);
…
PluginRegistry.stopPlugin(receiver);
```

Develop Your Own Plugins!

- Write your own class that implements the Plugin interface
  - The activate() method called to start the plugin.
- Extend the PluginSkeleton abstract class

Receivers

- Opposite of appenders
  - Appenders “export” logging events outside of the log4j environment.
  - Receivers “import” logging events into the log4j environment.
- Implemented as a plugin.



Plethora of Receivers

- SocketReceiver
  - matches SocketAppender
- SocketHubReceiver
  - matches SocketHubAppender
- JMSReceiver
  - matches JMSAppender
- SocketServer, JMSSink now obsolete
  - Configure all of the above in configuration file, mixing and matching as needed.

Develop Your Own Receivers!

- Subclass Receiver class
- Define implementation to “receive” logging events
- Call doPost(LoggingEvent) method to “post” event into log4j environment.
- Other receivers implemented (with matching appenders):
  - MulticastReceiver
  - UDPReceiver

Watchdogs

- “Watches” a source and reconfigures the log4j environment when the source “changes”.
- Deprecates the configureAndWatch() method used in current Configurators
  - Only supported configuration files
  - Not much control over starting and stopping
- Watchdogs implemented as plugins

Watchdogs Continued

- Will support any Configurator class for reconfiguration
  - Configurator class should support plugins
  - Configuration file can contain Watchdogs
- Support many different types of “sources”:
  - Time based watchdogs – modification time
    - FileWatchdog – watches a configuration file
    - HttpWatchdog – watches a url (LastModified)
  - Push based watchdogs
    - SocketWatchdog – new configuration passed in

Develop Your Own Watchdogs!

- Subclass Watchdog class
- Implement checkForReconfiguration() method
  - Returns true if source has changed
- Implement reconfigure() method
  - Implement to use specific source to configure
- Some helper routines in Watchdog class
  - reconfigureByURL
  - reconfigureByInputStream

Interoperability with log4j-like frameworks in other languages

- Ultimate goal: Logging across distributed processes implemented in various languages and/or environments
- XML description of LoggingEvent
  - LoggingEvent instances not easily serializable across languages or environments
  - XMLLayout outputs XML
  - XMLDecoder decodes received XML

Interoperability Support

- log4net (.NET) 1.2.0 beta 8
  - Events transportable to log4j using UDPAppender and XMLLayout (log4j.dtd).
  - log4j configured to use UDPReceiver and XMLLayout to receive
- log4cxx (C++)
  - Events transportable to log4j using XMLSocketAppender and XMLLayout
  - Log4j configured to use XMLSocketReceiver and XMLLayout to receive
- log4perl (perl)
  - log4perl XMLAppender outputs events in XML

Interoperability Support

- JDK 1.4 Logging
  - Events transportable to log4j using SocketHandler and XMLFormatter (logger.dtd)
  - log4j configured with SocketReceiver and XMLLayout to receive
- JDBCReciever (in log4j-sandbox)
  - Suitable for environments without log4j-like framework

Chainsaw v2

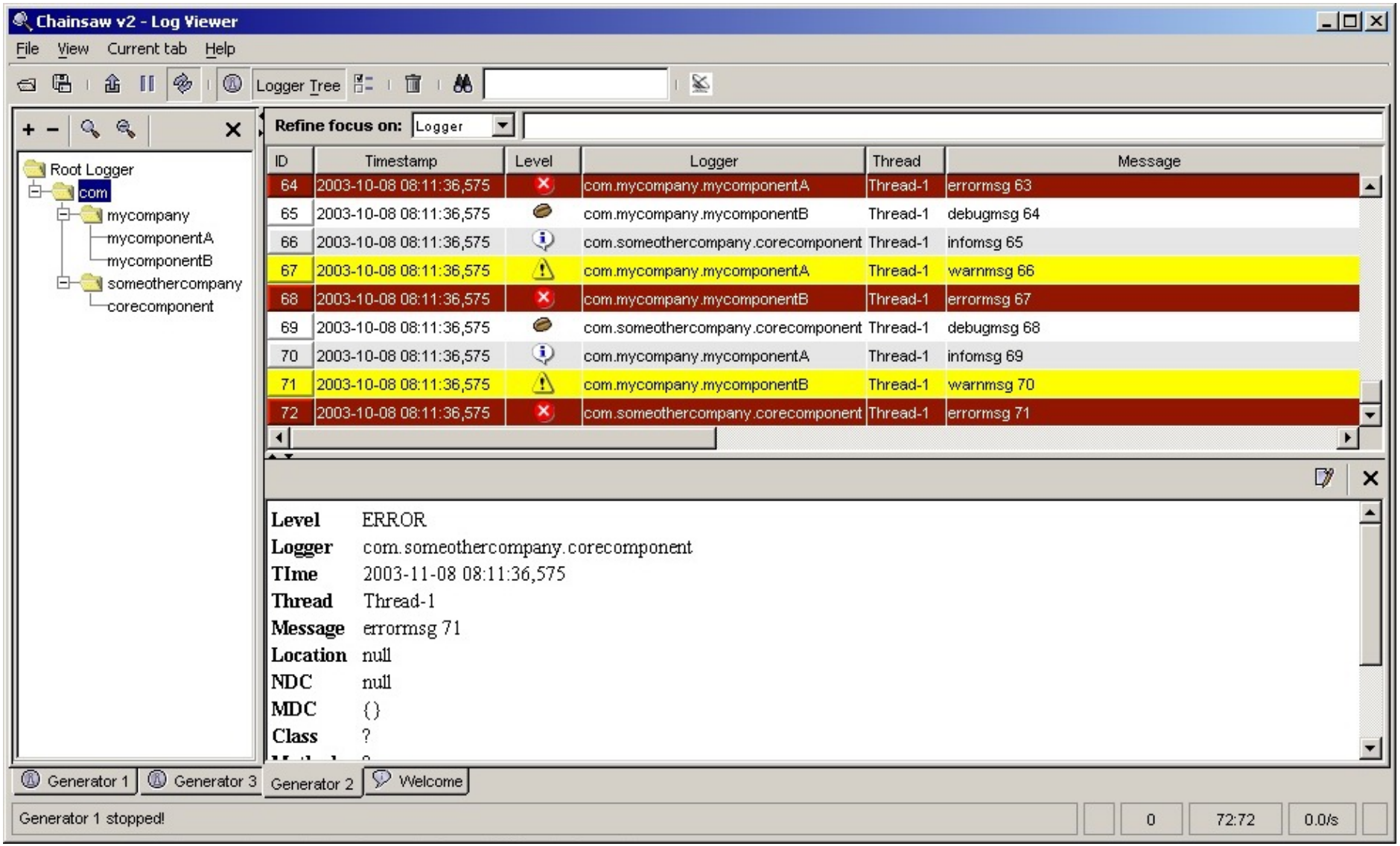
- Practically re-architected from the ground up
- Focus on usability, ease of use
- Puts more power in the hands of users
- Closer integration with log4j core functionality

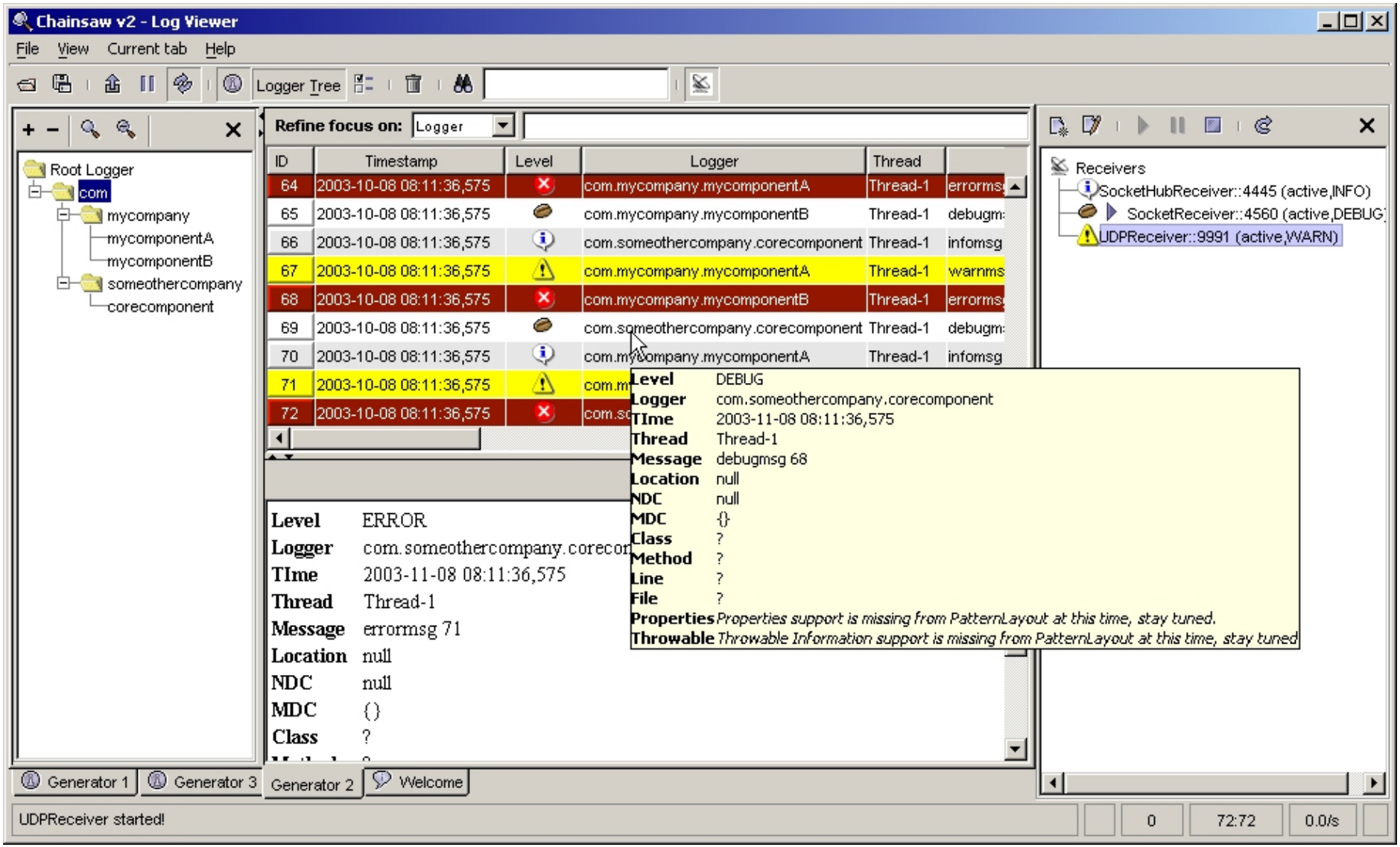
Chainsaw v2 Features

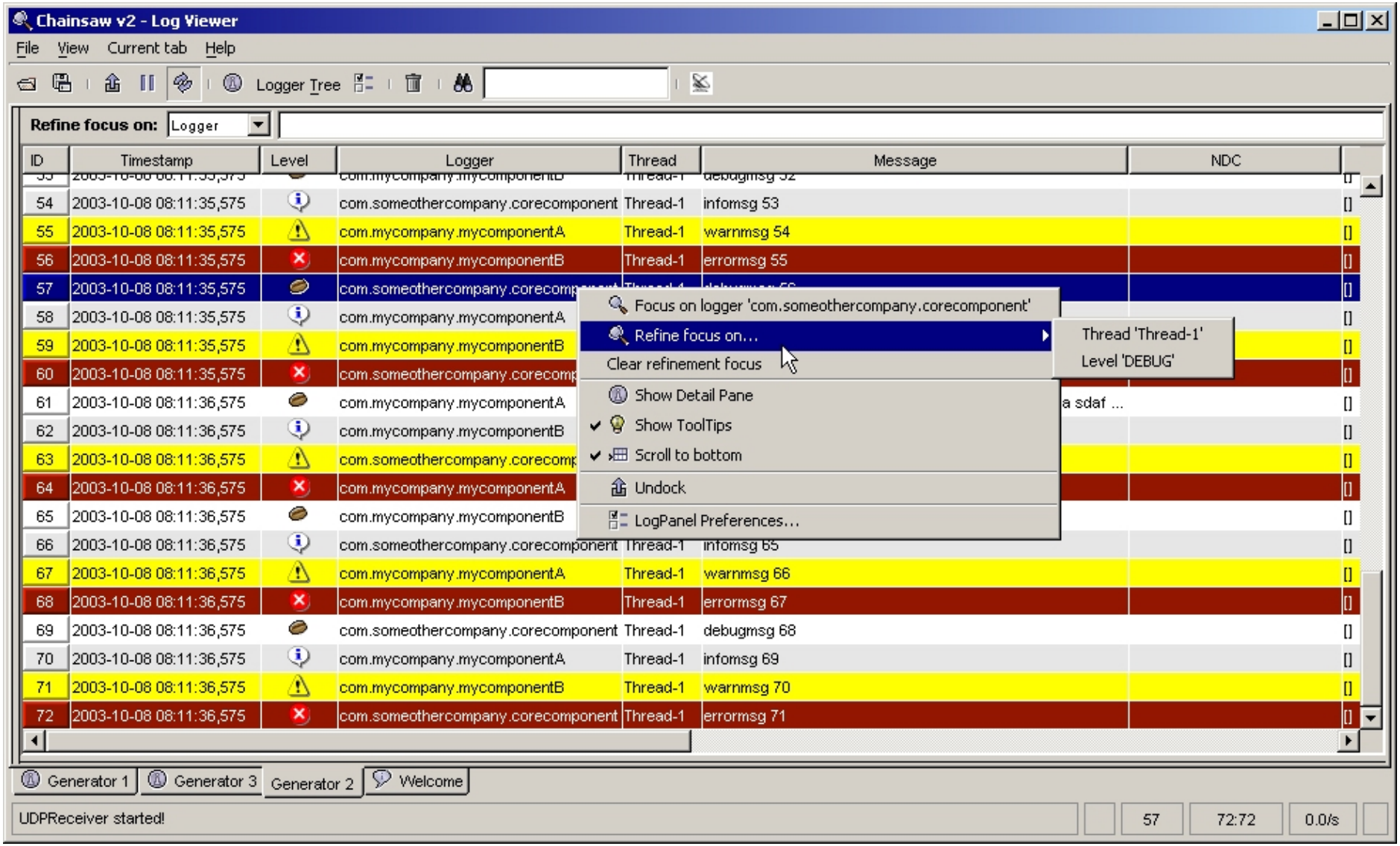
- Cyclic buffer option
  - no more OutOfMemoryErrors
- If you can Receive(r) it, you can view it
- Colouring of rows
- Undockable features
  - application specific logs can be in their own window for ease of use
- Extremely Customizable
  - Per-Panel preferences saved, remembered for the next session
- Built-in Docs and tutorial

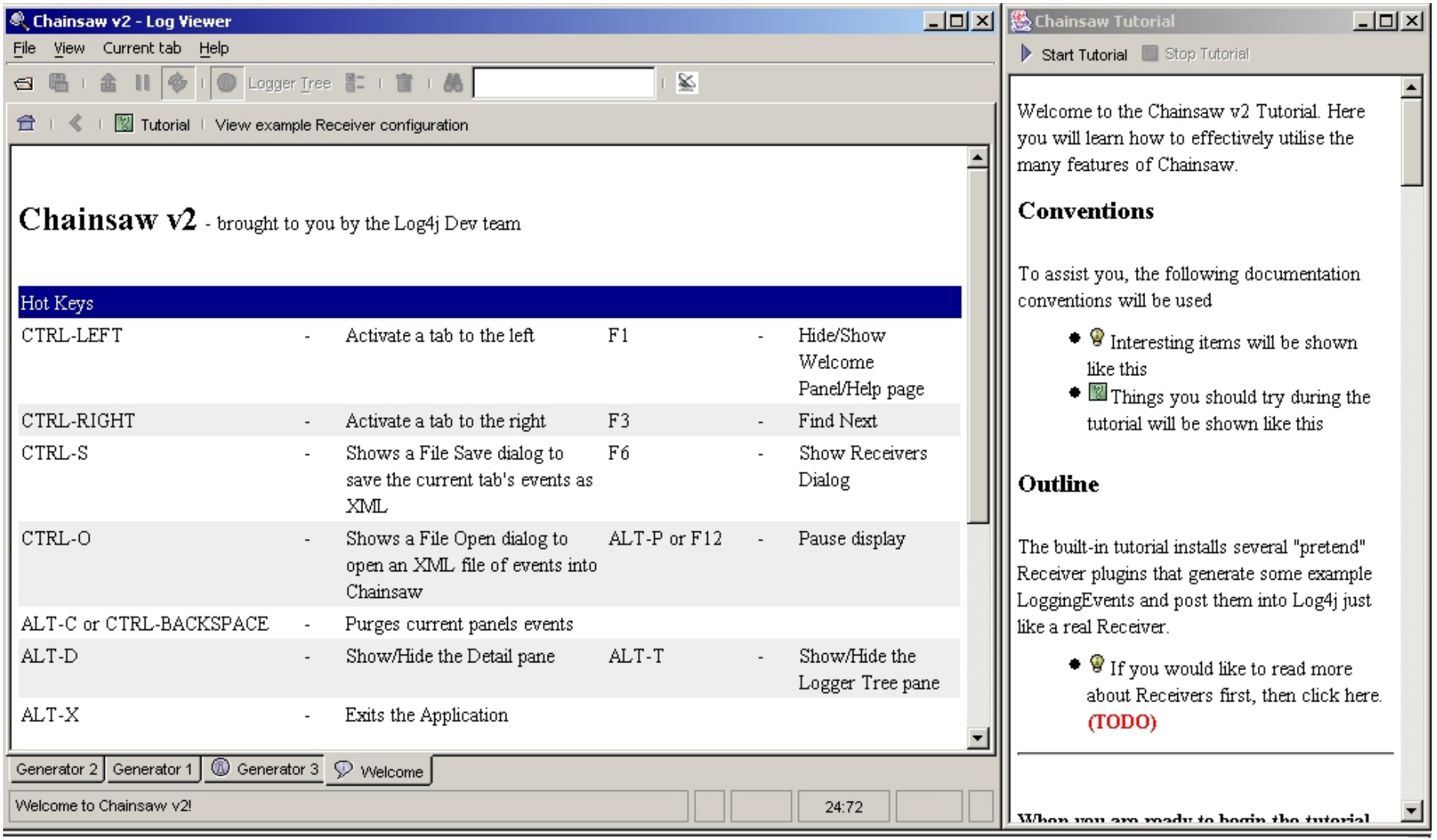
Chainsaw v2 - More to Come…

- Expression based filtering
  - define a filter rule in the form
    Level > ERROR || Message LIKE '%blah% '
  - provide a full expression editor with a log4j event 'language', and IDE-like auto-completion (similar to editing Java source files in Eclipse).
- Suggestions? Feedback welcome!


Chainsaw v2 - Log Viewer
File View Current tab Help
Logger Tree
Root Logger
com
mycompany
mycomponentA
mycomponentB
someothercompany
corecomponent
Refine focus on: Logger
ID Timestamp Level Logger Thread Message
64 2003-10-08 08:11:36,575 com.mycompany.mycomponentA Thread-1 errormsg 63
65 2003-10-08 08:11:36,575 com.mycompany.mycomponentB Thread-1 debugmsg 64
66 2003-10-08 08:11:36,575 com.someothercompany.corecomponent Thread-1 infomsg 65
67 2003-10-08 08:11:36,575 com.mycompany.mycomponentA Thread-1 warnmsg 66
68 2003-10-08 08:11:36,575 com.mycompany.mycomponentB Thread-1 errormsg 67
69 2003-10-08 08:11:36,575 com.someothercompany.corecomponent Thread-1 debugmsg 68
70 2003-10-08 08:11:36,575 com.mycompany.mycomponentA Thread-1 infomsg 69
71 2003-10-08 08:11:36,575 com.mycompany.mycomponentB Thread-1 warnmsg 70
72 2003-10-08 08:11:36,575 com.someothercompany.corecomponent Thread-1 errormsg 71
Level ERROR
Logger com.someothercompany.corecomponent
TIme 2003-11-08 08:11:36,575
Thread Thread-1
Message errormsg 71
Location null
NDC null
MDC {}
Class ?
Generator 1
Generator 3
Generator 2
Welcome
Generator 1 stopped!
0
72:72
0.0/s


Chainsaw v2 - Log Viewer
File View Current tab Help
Logger Tree
Refine focus on: Logger
ID Timestamp Level Logger Thread
64 2003-10-08 08:11:36,575 com.mycompany.mycomponentA Thread-1 errormsg
65 2003-10-08 08:11:36,575 com.mycompany.mycomponentB Thread-1 debugm
66 2003-10-08 08:11:36,575 com.someothercompany.corecomponent Thread-1 infomsg
67 2003-10-08 08:11:36,575 com.mycompany.mycomponentA Thread-1 warnms
68 2003-10-08 08:11:36,575 com.mycompany.mycomponentB Thread-1 errormsg
69 2003-10-08 08:11:36,575 com.someothercompany.corecomponent Thread-1 debugm
70 2003-10-08 08:11:36,575 com.mycompany.mycomponentA Thread-1 infomsg
71 2003-10-08 08:11:36,575
72 2003-10-08 08:11:36,575
Root Logger
com
mycompany
mycomponentA
mycomponentB
someothercompany
corecomponent
Receivers
SocketHubReceiver::4445 (active,INFO)
SocketReceiver::4560 (active,DEBUG)
UDPReceiver::9991 (active,WARN)
Level ERROR
Logger com.someothercompany.corecon
TIme 2003-11-08 08:11:36,575
Thread Thread-1
Message errormsg 71
Location null
NDC null
MDC {}
Class ?
Level DEBUG
Logger com.someothercompany.corecomponent
TIme 2003-11-08 08:11:36,575
Thread Thread-1
Message debugmsg 68
Location null
NDC null
MDC {}
Class ?
Method ?
Line ?
File ?
Properties Properties support is missing from PatternLayout at this time, stay tuned.
Throwable Throwable Information support is missing from PatternLayout at this time, stay tuned
Generator 1
Generator 3
Generator 2
Welcome
UDPReceiver started!
0
72:72
0.0/s


Chainsaw v2 - Log Viewer
File View Current tab Help
Logger Tree
Refine focus on: Logger
ID Timestamp Level Logger Thread Message NDC
54 2003-10-08 08:11:35,575 com.someothercompany.corecomponent Thread-1 infomsg 53 []
55 2003-10-08 08:11:35,575 com.mycompany.mycomponentA Thread-1 warnmsg 54 []
56 2003-10-08 08:11:35,575 com.mycompany.mycomponentB Thread-1 errormsg 55 []
57 2003-10-08 08:11:35,575 com.someothercompany.corecomp []
58 2003-10-08 08:11:35,575 com.mycompany.mycomponentA []
59 2003-10-08 08:11:35,575 com.mycompany.mycomponentB []
60 2003-10-08 08:11:35,575 com.someothercompany.corecomp []
61 2003-10-08 08:11:36,575 com.mycompany.mycomponentA a sdaf ... []
62 2003-10-08 08:11:36,575 com.mycompany.mycomponentB []
63 2003-10-08 08:11:36,575 com.someothercompany.corecomp []
64 2003-10-08 08:11:36,575 com.mycompany.mycomponentA []
65 2003-10-08 08:11:36,575 com.mycompany.mycomponentB []
66 2003-10-08 08:11:36,575 com.someothercompany.corecomponent Thread-1 infomsg 65 []
67 2003-10-08 08:11:36,575 com.mycompany.mycomponentA Thread-1 warnmsg 66 []
68 2003-10-08 08:11:36,575 com.mycompany.mycomponentB Thread-1 errormsg 67 []
69 2003-10-08 08:11:36,575 com.someothercompany.corecomponent Thread-1 debugmsg 68 []
70 2003-10-08 08:11:36,575 com.mycompany.mycomponentA Thread-1 infomsg 69 []
71 2003-10-08 08:11:36,575 com.mycompany.mycomponentB Thread-1 warnmsg 70 []
72 2003-10-08 08:11:36,575 com.someothercompany.corecomponent Thread-1 errormsg 71 []
Focus on logger 'com.someothercompany.corecomponent'
Refine focus on...
Clear refinement focus
Show Detail Pane
Show ToolTips
Scroll to bottom
Undock
LogPanel Preferences...
Thread 'Thread-1'
Level 'DEBUG'
Generator 1
Generator 3
Generator 2
Welcome
UDPReceiver started!
57
72:72
0.0/s


Chainsaw v2 - Log Viewer
File View Current tab Help
Logger Tree
Tutorial View example Receiver configuration
Chainsaw v2 - brought to you by the Log4j Dev team
Hot Keys
CTRL-LEFT - Activate a tab to the left
CTRL-RIGHT - Activate a tab to the right
CTRL-S - Shows a File Save dialog to save the current tab's events as XML
CTRL-O - Shows a File Open dialog to open an XML file of events into Chainsaw
ALT-C or CTRL-BACKSPACE - Purges current panels events
ALT-D - Show/Hide the Detail pane
ALT-X - Exits the Application
F1 - Hide/Show Welcome Panel/Help page
F3 - Find Next
F6 - Show Receivers Dialog
ALT-P or F12 - Pause display
ALT-T - Show/Hide the Logger Tree pane
Generator 2
Generator 1
Generator 3
Welcome
Welcome to Chainsaw v2!
24:72
Chainsaw Tutorial
Start Tutorial
Stop Tutorial
Welcome to the Chainsaw v2 Tutorial. Here you will learn how to effectively utilise the many features of Chainsaw.
Conventions
To assist you, the following documentation conventions will be used
Interesting items will be shown like this
Things you should try during the tutorial will be shown like this
Outline
The built-in tutorial installs several "pretend" Receiver plugins that generate some example LoggingEvents and post them into Log4j just like a real Receiver.
If you would like to read more about Receivers first, then click here. (TODO)

Chainsaw v2 Demo



Varia

- Notifications of internal events
- Improved JMX support
- JDK 1.2 or higher required
- Selectors
  - ContextClassLoaderSelector, ContextJNDISelector
- Servlet related
  - InitServlet, ConfigurationServlet, MDCFilter
- Suggestions?

Join Us!

- http://jakarta.apache.org/log4j
- Mailing lists
  - log4j-user@jakarta.apache.org
  - log4j-dev@jakarta.apache.org
- IRC
  - irc://irc.freenode.net:6667/#apache-log4j
- log4j-sandbox cvs
  - “Experimental” submissions
    - Chainsaw v2 (started there)
    - Selectors, servlet, etc

This presenation has ended.
Click anywhere to return.

_

